CESDE

Santiago Yosa

Ingeniero Sistemas de Información

CESDE | comfama

Unidad II

HERENCIA POO

¿Qué es herencia?

Llamada a métodos superiores: palabra reservada SUPER

SISTEMA CONTROL DE VERSIONES

-Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Situación

Tienes una clase base CuentaBancaria con un método retirar(monto) que simplemente resta el dinero del saldo. El banco te pide crear una CuentaCredito. Esta nueva cuenta también permite retirar dinero, pero tiene una política extra: *por cada retiro, se cobra una comisión de $5*.

- **El enfoque novato (Copy-Paste):** Copiar toda la lógica de validación de saldo de la clase base y pegarla en la clase hija, añadiendo la resta de los $5. Esto destruye la mantenibilidad.

- **El enfoque arquitectónico (super):** La clase hija dice: "Yo haré mi parte (cobrar los $5) y luego le diré a mi Padre que haga el resto del trabajo pesado (validar y restar el saldo)". Aquí es donde entra la llamada a métodos superiores.

La Sobrescritura de Métodos (@Override)

Cuando un hijo hereda de un padre, hereda sus métodos. Pero a veces, el hijo necesita hacer la misma acción de una forma diferente. A esto se le llama **Sobrescribir**.

- **Regla técnica:** El método en el hijo debe tener **exactamente la misma firma** (mismo nombre, mismos parámetros, mismo tipo de retorno) que el método en el padre.
- **La Anotación @Override:** Se coloca justo arriba del método en el hijo. No es obligatoria para que el código compile, pero es **exigida profesionalmente**. Le dice al compilador: "Verifica que realmente estoy aplastando un método del padre, si me equivoco en una letra, márcame error".

El rescate del Padre: super.metodo()

Cuando sobrescribes un método, el comportamiento del padre queda "aplastado" (oculto) para esa clase hija. Si necesitas reutilizar la lógica original del padre dentro del nuevo método del hijo, usas la palabra reservada super.

- super() (con paréntesis): Llama al **constructor** del padre. Solo va en la primera línea del constructor del hijo.
- super.nombreDelMetodo(): Llama a un **comportamiento** del padre. Puede ir en *cualquier línea* del nuevo método.

Flujo de Trabajo Integrado con Git

En esta etapa, el código deja de ser un experimento desechable. Cada unidad lógica completada debe ser un *commit*.

- **Paso 1:** Crear y probar la Clase Base -> git commit -m "feat: crear clase base CuentaBancaria"
- **Paso 2:** Crear la Clase Hija y sobrescribir -> git commit -m "feat: implementar CuentaCredito con comision usando super"

Demo

```
public class Documento {
   protected String contenido;

   public void imprimir() {
      System.out.println("Imprimiendo en blanco y negro: " + contenido);
   }
}

// CLASE HIJA
public class DocumentoConfidencial extends Documento {

   @Override // Buena práctica obligatoria
   public void imprimir() {
      // 1. Lógica exclusiva del hijo
      System.out.println("--- ADVERTENCIA: MATERIAL CLASIFICADO ---");

      // 2. Invocación de la lógica del padre
      super.imprimir();

      // 3. Más lógica del hijo
      System.out.println("--- DESTRUIR DESPUÉS DE LEER ---");
   }
}
```

La Palabra Reservada super:

¿Qué es? Es una referencia directa a la superclase inmediata. Permite acceder a miembros (campos y métodos) del padre y, más importante, a su constructor.

Uso 1: super() en Constructores (Regla de Oro)

El Problema: Un objeto de la subclase es también un objeto de la superclase. Antes de que la subclase pueda inicializar sus propios campos, la parte "base" del objeto debe estar completamente construida.

La Solución/Regla: La llamada al constructor de la superclase, super(...), **debe ser la primera sentencia** en el constructor de la subclase. Si no escribes una llamada explícita a super(...), Java insertará una llamada implícita a super() (el constructor sin argumentos del padre). Si el padre no tiene un constructor sin argumentos, el código **no compilará**.

Uso 2: super.metodo() para Extender Comportamiento

El Problema: Quieres redefinir un método en la subclase (esto se llama **sobrescritura de método** o *method overriding*), pero necesitas invocar la lógica original del padre como parte de la nueva lógica.

La Solución: Usas la anotación @Override para indicar al compilador tu intención (buena práctica que previene errores). Dentro del método, usas super.nombreDelMetodo() para ejecutar la versión de la superclase.

¿Qué es Sobrescribir un Método (Override)?

Sobrescribir un método significa **crear una nueva versión de un método en una clase hija que ya existía en la clase padre**.

Piensa en nuestro ejemplo del Vehiculo.

- La clase Vehiculo tiene un método obtenerEstado().
- La clase Coche también tiene un método obtenerEstado().

Como Coche es una subclase de Vehiculo, decimos que el obtenerEstado de Coche **sobrescribe** al obtenerEstado de Vehiculo. Cuando llamas a este método en un objeto Coche, se ejecuta la versión del Coche, no la del Vehiculo.

¿Y qué es la etiqueta @Override?
@Override es una **anotación**. Es una etiqueta especial que le pones a un método para decirle al compilador de Java:

"Oye, compilador. Mi intención con este método es **sobrescribir** un método que existe en la clase padre. Por favor, verifica que lo estoy haciendo bien."

¿Por qué es una herramienta tan importante?
Su principal función es **prevenir errores**. Es tu red de seguridad.

Reglas Clave para Sobrescribir
Para que un método sea una sobrescritura válida (y para que @Override no dé error), debe cumplir estas reglas:

1. **Mismo Nombre:** El nombre del método en la clase hija debe ser idéntico al de la clase padre.
2. **Mismos Parámetros:** Debe tener exactamente la misma cantidad y tipo de parámetros en el mismo orden.
3. **No puede ser más restrictivo:** No puedes hacer un método public en el padre y private en el hijo. Sí puedes hacerlo menos restrictivo (de protected a public, por ejemplo).

Ejemplos

```
// Archivo: Figura.java
public class Figura {
    protected String color; // protected para ser accesible por las subclases

    public Figura(String color) {
        System.out.println("Constructor de Figura ejecutándose...");
        this.color = color;
    }

    public String describir() {
        return "Soy una figura de color " + this.color + ".";
    }
}

// Archivo: Circulo.java
public class Circulo extends Figura {
    private double radio; // private, solo de Circulo

    public Circulo(String color, double radio) {
        // La llamada a super() DEBE ser la primera línea
        super(color);
        System.out.println("Constructor de Circulo ejecutándose...");
        this.radio = radio;
    }

    @Override // Buena práctica: anotar la sobrescritura
    public String describir() {
        String descripcionBase = super.describir(); // Llama al método del padre
        return descripcionBase + " Además, soy un círculo con un radio de " + this.radio + ".";
    }
}

// Archivo: Main.java (Para probar)
public class Main {
    public static void main(String[] args) {
        Circulo miCirculo = new Circulo("rojo", 10.0);
        System.out.println("--------------------");
        System.out.println(miCirculo.describir());
    }
```

Ejemplo 2

```
// Archivo: Vehiculo.java
public class Vehiculo {
    protected String marca;
    protected int velocidad;

    // Constructor de la clase base
    public Vehiculo(String marca) {
        this.marca = marca;
        this.velocidad = 0; // Un vehículo nuevo empieza detenido
        System.out.println("-> Constructor de Vehiculo ejecutado.");
    }

    public void acelerar(int cantidad) {
        this.velocidad += cantidad;
        System.out.println("El vehículo " + this.marca + " acelera.");
    }

    public String obtenerEstado() {
        return "Marca: " + this.marca + ", Velocidad: " + this.velocidad + " km/h";
    }
}
```

```java
// Archivo: Coche.java
public class Coche extends Vehiculo {
    private String modelo;

    // Constructor de la subclase
    public Coche(String marca, String modelo) {
        // 1. LLAMADA AL CONSTRUCTOR PADRE (OBLIGATORIA)
        // Le pasamos la 'marca' que el constructor de Vehiculo necesita.
        super(marca);

        this.modelo = modelo;
        System.out.println("-> Constructor de Coche ejecutado.");
    }

    // 2. SOBRESCRITURA Y EXTENSIÓN DE UN MÉTODO
    @Override
    public String obtenerEstado() {
        // Llamamos al método original del padre para no repetir código
        String estadoBase = super.obtenerEstado();

        // Añadimos la información específica del coche
        return estadoBase + ", Modelo: " + this.modelo;
    }
}
```

```java
// Archivo: Motocicleta.java
public class Motocicleta extends Vehiculo {
    private int cilindrada; // Atributo específico de Motocicleta

    // Constructor de la subclase
    public Motocicleta(String marca, int cilindrada) {
        // 1. LLAMADA OBLIGATORIA AL CONSTRUCTOR DEL PADRE
        // Le pasamos la 'marca' que el constructor de Vehiculo necesita.
        super(marca);

        this.cilindrada = cilindrada;
        System.out.println("-> Constructor de Motocicleta ejecutado.");
    }

    // 2. MÉTODO ÚNICO DE ESTA SUBCLASE
    public void hacerWilly() {
        System.out.println("¡La motocicleta " + this.marca + " está haciendo un willy!");
    }

    // 3. SOBRESCRITURA Y EXTENSIÓN DE UN MÉTODO DEL PADRE
    @Override
    public String obtenerEstado() {
        // Reutilizamos la lógica del padre para obtener la información base
        String estadoBase = super.obtenerEstado();

        // Y le añadimos los detalles específicos de la motocicleta
        return estadoBase + ", Cilindrada: " + this.cilindrada + "cc";
    }
}
```

```java
// Archivo: Main.java
public class Main {
    public static void main(String[] args) {
        // --- Creamos y usamos un Coche ---
        Coche miCoche = new Coche("Toyota", "Corolla");
        System.out.println(miCoche.obtenerEstado());
        miCoche.acelerar(100);
        System.out.println(miCoche.obtenerEstado());

        System.out.println("\n=====================================\n");

        // --- Creamos y usamos una Motocicleta ---
        Motocicleta miMoto = new Motocicleta("Honda", 600);
        System.out.println(miMoto.obtenerEstado());
        miMoto.acelerar(120);
        miMoto.hacerWilly(); // Llamamos a su método único
        System.out.println(miMoto.obtenerEstado());
    }
}
```

Git y GitHub: Tu Red de Seguridad Profesional

El .gitignore para Java: En un proyecto Java, es crucial ignorar los archivos generados por el compilador y el IDE. Un buen .gitignore para Java debería incluir:

```
# Compiled class files
*.class
# Build directories
/bin/
/build/
/target/

# IDE files
.idea/
*.iml
```

Práctica Acompañada

Tu Misión: Modelar diferentes tipos de publicaciones en una biblioteca.

Crea una rama main. Inicia el proyecto con la clase principal App. Y Luego crea una rama feature/superclase

Construye la Superclase Publicacion

1. Crea un archivo Publicacion.java.
2. Define una clase public class Publicacion con dos campos protected: String titulo y String autor.
3. Crea un constructor que reciba e inicialice estos dos campos.
4. Añade un método public void mostrarFicha() que imprima los detalles.

Versiona – Suber cambios al repositorio remoto

Crea una rama feature/libro

Crea la Subclase Libro

1. Crea un archivo Libro.java.
2. Define public class Libro extends Publicacion.
3. Añade un campo private int numPaginas.
4. Crea un constructor para Libro que acepte título, autor y número de páginas.
5. La **primera línea** debe ser super(titulo, autor);.
6. Luego, inicializa this.numPaginas.

Versiona – Suber cambios al repositorio remoto

Sobrescribe y Extiende en Libro

1. En Libro, sobrescribe mostrarFicha(). Usa la anotación @Override.
2. La primera línea del método debe ser super.mostrarFicha();.
3. Añade un System.out.println para mostrar el número de páginas.

Versiona – Suber cambios al repositorio remoto

Prueba tu Código

En el archivo App.java.
Dentro de public static void main(String[] args):

Crea una instancia de Libro:

```
Libro miLibro = new Libro("El Principito", "Antoine de Saint-Exupéry", 92);
```

Llama al método: miLibro.mostrarFicha();

Súbelo a GitHub y realiza un merge de las ramas.

Learning by Doing

Ejemplo de Aprendizaje (El E-Commerce)

Instrucciones para el estudiante:

1. Abre tu terminal, inicializa un repositorio (git init) y crea un archivo .gitignore.
2. Crea la clase Producto con atributos nombre y precio. Crea el método calcularTotal() que retorne el precio aplicándole un 19% de IVA (precio * 1.19).
3. Haz un commit: feat: logica base de producto e impuestos.
4. Crea la clase ProductoImportado que herede de Producto. Añade el atributo impuestoAduana.
5. Sobrescribe (@Override) el método calcularTotal(). Llama al método del padre usando super.calcularTotal() y súmale el impuestoAduana.
6. Haz un commit: feat: aplicacion de aduanas a productos importados.
7. En el main, imprime el total de un producto normal y uno importado. Sube a GitHub.

```java
public class Producto {
    protected String nombre;
    protected double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public double calcularTotal() {
        return precio * 1.19; // IVA estándar
    }
}

public class ProductoImportado extends Producto {
    private double impuestoAduana;

    public ProductoImportado(String nombre, double precio, double impuestoAduana) {
        super(nombre, precio);
        this.impuestoAduana = impuestoAduana;
    }

    @Override
    public double calcularTotal() {
        // Reutilizamos el cálculo del IVA y le sumamos la aduana
        return super.calcularTotal() + impuestoAduana;
    }
}
```

CESDE | comfama